Memory Slice (.msl)

Binary Format Specification

Version: 1.0.0
Status: Working Draft

Abstract

Memory Slice (`.msl`) is a self-describing, block-based binary format for capturing the forensic state of a single operating system process. It records both the virtual address space (with per-page acquisition status) and transient OS-queryable metadata that exists only while the process is alive. This document specifies the binary layout: file header, block architecture, integrity chain, capture-time payloads, cross-referencing, capability bitmap, and raw dump import mechanism.

This specification accompanies the research paper *Memory Slice: A Process-Centric Dump Format Enabling Differential Structure Discovery in Memory Forensics*.

Contents

1 Scope and Conventions **2**
- 1.1 Scope . . . 2
- 1.2 Normative Language . . . 2
- 1.3 Encoding Conventions . . . 2
- 1.4 UUID Requirements . . . 2
- 1.5 Terminology . . . 2

2 Format Overview **4**
- 2.1 Design Principles . . . 4
- 2.2 Scope of the Integrity Guarantee . . . 4

3 File Header **5**
- 3.1 Endianness Bootstrapping . . . 6
- 3.2 Version Compatibility . . . 7

4 Block Architecture **8**
- 4.1 Common Block Header (80 bytes) . . . 8
- 4.2 Block Flags . . . 9
- 4.3 Block Type Registry . . . 9
- 4.4 Integrity Chain (`PrevHash`) . . . 10
- 4.5 End-of-Capture Block (`0x0FFF`) . . . 10

5 Capture-Time Payloads **12**
- 5.1 Memory Region (`0x0001`) . . . 12
- 5.2 Module Entry (`0x0002`) . . . 14
- 5.3 Module List Index (`0x0010`) . . . 15
- 5.4 Process Identity (`0x0040`) . . . 16
- 5.5 Related Dump (`0x0041`) . . . 16
- 5.6 Key Hint (`0x0020`) . . . 17

6 Three-State Virtual Address Map **18**

7 Block Cross-Referencing **19**

8 Capability Bitmap **21**

9 Raw Dump Import **22**
- 9.1 Import Provenance Block (`0x0030`) . . . 22
- 9.2 Import Procedure . . . 22

10 Parsing Walkthrough **24**

11 Worked Example **25**

12 Conformance **26**
- 12.1 Producer . . . 26
- 12.2 Consumer . . . 26

1 Scope and Conventions

1.1 Scope

This document defines the binary wire format of Memory Slice (`.msl`) files. It does *not* specify acquisition procedures or analysis algorithms.

1.2 Normative Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, "RECOMMENDED", **MAY**, and **OPTIONAL** are per RFC 2119 [1].

1.3 Encoding Conventions

- The file header's `Endianness` byte at offset `0x08` determines the byte order for all multi-byte integers throughout the file. `0x01` = little-endian (default); `0x02` = big-endian. Producers **SHOULD** use little-endian unless the target platform is natively big-endian. Because this is a single `uint8`, it has no endianness ambiguity (Section 3.1).
- Strings are UTF-8, null-terminated, padded to **8-byte** boundaries with zero bytes. $\text{pad8}(n) = \lceil n/8 \rceil \times 8$. All padding bytes inserted to reach alignment boundaries **MUST** be initialized to `0x00`. This prevents tools from leaking sensitive heap contents into padding space.
- Variable-length fields (`PageStateMap`, `NativeBlob`) are padded to 8-byte boundaries with zero bytes.
- UUIDs **MUST** be version 4 (random) per RFC 4122 [2], stored as 16 raw bytes in RFC 4122 network byte order (big-endian sub-fields), regardless of the file's `Endianness`. See Section 1.4.
- Hashes use BLAKE3 [3] with 32-byte output (Section 4.4).
- Timestamps: unsigned 64-bit nanoseconds since Unix epoch (1970-01-01T00:00:00Z).
- All `Reserved` fields **MUST** be zero (producers) and **MUST** be ignored (consumers).

1.4 UUID Requirements

All UUID fields (`DumpUUID`, `BlockUUID`, `ParentUUID`, payload UUIDs) **MUST** be version 4 (random) per RFC 4122. The 4-bit version field (bits 48–51) **MUST** be `0100`; the 2-bit variant (bits 64–65) **MUST** be `10`. UUIDs are always in RFC 4122 binary layout regardless of the file's endianness.

Implementation guidance: UUID generation performance

UUID generation can bottleneck high-throughput acquisition. Producers need not use a CSRNG; UUIDs are identifiers, not security tokens. A thread-local fast PRNG such as `xoshiro256++` or `PCG64`, seeded once from OS entropy, provides sufficient uniqueness ($\sim 2^{-61}$ collision probability per pair) while avoiding CSRNG overhead per block.

1.5 Terminology

Producer
: Software that creates or appends blocks.

Consumer
Software that reads an MSL file.

Acquirer
A producer capturing live process state (types `0x0000`–`0x0FFF`).

Importer
A producer converting a raw dump into MSL (Section 9).

Block
Fixed header + variable payload.

Payload
Block-type-specific data after the header.

The preceding section established the conventions, terminology, and encoding rules that apply throughout the specification. The following sections define the format itself, starting with the high-level file structure and then drilling into each component: the file header (Section 3), block architecture (Section 4), capture-time payloads (Section 5), the three-state page model (Section 6), cross-referencing (Section 7), and the capability bitmap (Section 8). Sections 9–12 cover raw dump import, parsing guidance, a worked example, and conformance requirements.

2 Format Overview

An MSL file is a linear byte stream: a **file header** (64 bytes in v1.0) followed by **typed, length-prefixed blocks**. No inter-block gaps. `HeaderSize` tells the consumer where blocks begin, enabling future header extensions. Figure 1 shows the high-level structure; Figure 2 shows a typical live-acquisition block sequence.

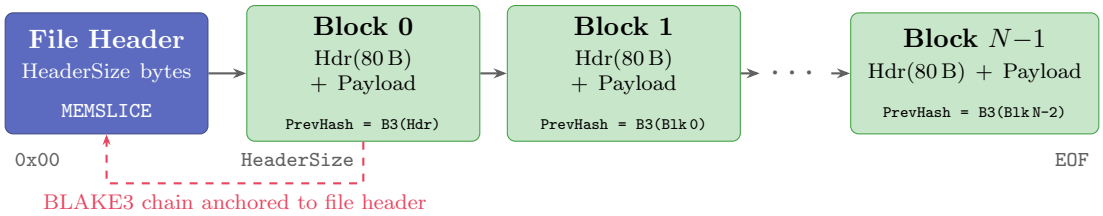

Figure 1: MSL file structure: a 64-byte file header followed by a linear sequence of BLAKE3-chained blocks.

Block 0's `PrevHash` $= BLAKE3$(File Header), anchoring the integrity chain. Each subsequent block hashes its predecessor, so modifying any block in the middle invalidates all subsequent hashes. Consumers detect end-of-file when fewer than 80 bytes remain or the next 4 bytes do not match the block magic (`0x4D534C43`).

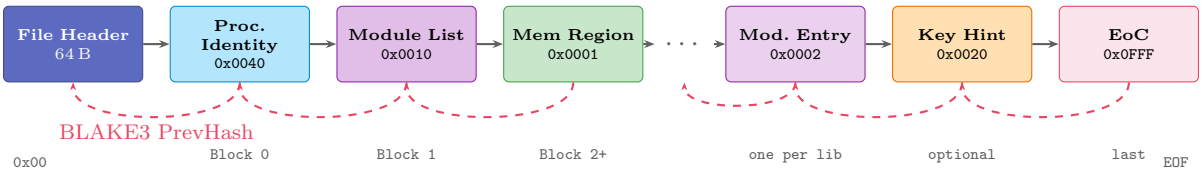

Figure 2: Typical live-acquisition block sequence. One Module Entry is emitted per loaded library (DLL, .so).

2.1 Design Principles

1. **Integrity Verification (Append-Only Policy, Anchored Hash Chain).** Producers treat all written blocks as append-only. The BLAKE3 chain from file header through all blocks enables consumers to detect accidental corruption or post-hoc modification (Section 2.2).
2. **Epistemic Honesty (Three-State Pages).** Captured vs. Failed vs. Unmapped.
3. **Self-Describing.** Endianness, version, header size, OS, arch, PID, capabilities.
4. **Dual-Layer OS Abstraction.** Normalized fields + OS-native opaque blob.

The header has no block count. Appending blocks would require mutating it, violating append-only integrity. Consumers read blocks sequentially until fewer than 80 bytes remain or the next 4 bytes do not match the block magic.

2.2 Scope of the Integrity Guarantee

The hash chain provides *tamper detection*, not *tamper resistance*. Because the chain uses an unkeyed hash function, any party with write access can modify a block and recompute all subsequent hashes. The chain therefore protects against **accidental corruption** (bit-rot, truncation,

transfer errors) and detects **naive modification** (altering a block without updating the chain). It does *not* protect against an adversary who deliberately rewrites the file.

For scenarios requiring tamper evidence admissible in legal or regulatory contexts, producers **SHOULD** complement the in-file hash chain with external mechanisms such as a digital signature (e.g., Ed25519) over the End-of-Capture `FileHash`, a timestamped hash commitment to a trusted third party (RFC 3161), or write-once storage media. In the absence of such mechanisms, the in-file hash chain alone **MUST NOT** be represented as providing forensic-grade tamper evidence.

3 File Header

The header (Table 1) places the single-byte `Endianness` at offset `0x08` (after the 8-byte magic) and the single-byte `HeaderSize` at `0x09`. A parser reads these two `uint8` values—which have no byte-order ambiguity—before encountering any multi-byte field.

Table 1: File header (64 bytes in version 1.0).

Offset	Size	Field	Description
0x00	8	Magic	0x4D454D534C494345 ("MEMSLICE"). Endianness-independent byte sequence.
0x08	1	Endianness	0x01=LE, 0x02=BE. Single byte: no endianness ambiguity. Invalid values **MUST** cause rejection.
0x09	1	HeaderSize	Header size in bytes (uint8). v1.0: **MUST** be 64 (0x40). First block starts here.
0x0A	2	Version	uint16 read per Endianness. The resulting integer encodes major in the high byte and minor in the low byte: version 1.0 is the integer 0x0100 (=256).
0x0C	4	Flags	Bit 0: Imported (file was created by importing a raw dump, not by live acquisition). Bits 1–31: reserved, zero.
0x10	8	CapBitmap	Capability bitmap (Section 8).
0x18	16	DumpUUID	UUIDv4 uniquely identifying this dump. Used for cross-file referencing (Related Dump blocks reference another file's DumpUUID), deduplication (identifying copies of the same capture), and case management.
0x28	8	Timestamp	Acquisition start (UTC wall-clock, nanoseconds since Unix epoch 1970-01-01T00:00:00Z).
0x30	2	OSType	Target OS (Table 2a).
0x32	2	ArchType	Target CPU (Table 2b).
0x34	4	PID	Process ID at acquisition.
0x38	1	ClockSource	Clock used for timestamps. 0x00=Unknown, 0x01=CLOCK_REALTIME (POSIX), 0x02=CLOCK_MONOTONIC_RAW (Linux), 0x03=QueryPerformanceCounter (Windows), 0x04=mach_absolute_time (macOS). 0x05–0xFE: reserved. 0xFF=Other. Producers **SHOULD** use a monotonic source for per-region timestamps; Timestamp in this header **SHOULD** be UTC wall-clock time.
0x39	7	Reserved	**MUST** be zero.

Size: $8 + 1 + 1 + 2 + 4 + 8 + 16 + 8 + 2 + 2 + 4 + 1 + 7 = 64$.

3.1 Endianness Bootstrapping

This uses the ELF approach (e_ident[EI_DATA] [4]): a single byte whose value is always read identically regardless of host byte order.

Parse sequence: (1) Read 8-byte magic. (2) Read Endianness at 0x08. (3) Read HeaderSize at 0x09. (4) All subsequent multi-byte reads use the determined order.

Memory content within PageData is stored in the *target architecture's* native order; Endianness governs only format-level structural integers.

3.2 Version Compatibility

Unknown major version: **SHOULD** reject. Unknown minor of known major: **MAY** parse. Always use `HeaderSize` to locate Block 0.
Table 2 defines the registered codes for the `OSType` and `ArchType` header fields.

Table 2: Target platform codes: (a) Operating systems and (b) CPU architectures.

(a) OS type codes (`OSType`)

Code	OS
0x0000	Windows
0x0001	Linux
0x0002	macOS
0x0003	Android
0x0004	iOS / iPadOS
0x0005	FreeBSD
0x0006	NetBSD
0x0007	OpenBSD
0x0008	QNX
0x0009	Fuchsia
0x000A--0x00FF	*Reserved*
0x0100--0xFFFE	*Vendor*
0xFFFF	Unknown

(b) Architecture codes (`ArchType`)

Code	Architecture
0x0000	x86 (IA-32)
0x0001	x86_64 (AMD64)
0x0002	ARM64 (AArch64)
0x0003	ARM32 (AArch32)
0x0004	MIPS32
0x0005	MIPS64
0x0006	RISC-V RV32
0x0007	RISC-V RV64
0x0008	PPC32
0x0009	PPC64
0x000A	s390x
0x000B	LoongArch64
0x000C--0x00FF	*Reserved*
0x0100--0xFFFE	*Vendor*
0xFFFF	Unknown

The file header provides the bootstrapping context—endianness, version, capabilities, and the offset at which blocks begin. The next section defines the block structure that makes up the rest of the file: the Common Block Header shared by all blocks, the type registry, and the BLAKE3 integrity chain.

4 Block Architecture

Every block in an MSL file consists of a **fixed 80-byte Common Block Header** followed by a **variable-length payload**. The Common Block Header has the same layout regardless of block type, which enables forward compatibility: a consumer that does not recognize a block's type reads the `BlockLength` field and skips (`BlockLength` − 80) bytes to reach the next block. Consumers detect end-of-file when fewer than 80 bytes remain after the current block or the next 4 bytes do not match the block magic (`0x4D534C43`).

4.1 Common Block Header (80 bytes)

The header layout (Table 3, Figure 3) ensures that all fields of 8 bytes or larger start on 8-byte-aligned offsets, enabling zero-copy casting of UUIDs to `uint64_t` pairs and direct BLAKE3 digest comparison without unaligned memory access faults (critical on strict architectures such as ARM).

Table 3: Block header (80 bytes). All fields ≥ 8 B are 8-byte aligned.

Off	Size	Field	Description
0x00	4	Magic	0x4D534C43 ("MSLC").
0x04	2	BlockType	Payload selector (Section 4.3).
0x06	2	Flags	Per-block flags (Table 4).
0x08	4	BlockLength	Total size (hdr+payload). ≥ 80.
0x0C	2	PayloadVersion	Payload layout version for this `BlockType`. Default: `0x0001`. Consumers that encounter an unknown version for a known type **SHOULD** skip the block via `BlockLength`.
0x0E	2	Reserved	**MUST** be zero.
0x10	16	BlockUUID	UUIDv4 identifier. *8-byte aligned.*
0x20	16	ParentUUID	Parent UUID or zeros. *8-byte aligned.*
0x30	32	PrevHash	BLAKE3 of preceding element (Section 4.4). *8-byte aligned.*
0x50	var	Payload	`BlockLength` − 80 bytes. *8-byte aligned.*

Verification: $4 + 2 + 2 + 4 + 2 + 2 + 16 + 16 + 32 = 80$. Payload starts at `0x50`.

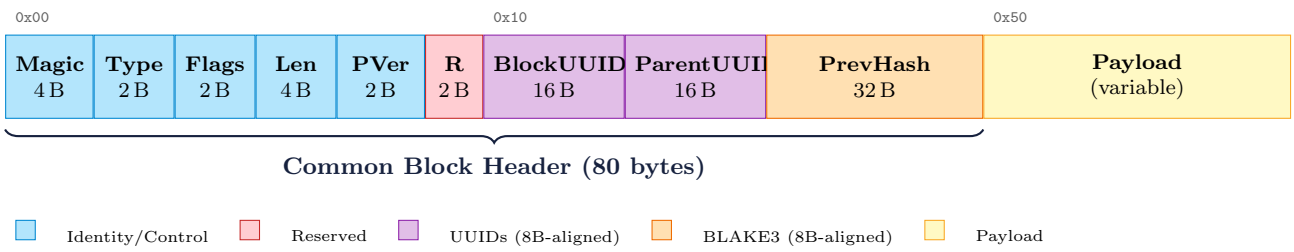

Figure 3: Common Block Header layout (80 bytes), shared by all block types. R=Reserved, PVer=PayloadVersion.

Every block begins with this 80-byte header (Figure 3). The `BlockType` selects the payload layout; `BlockLength` enables skipping unknown types; the `PrevHash` anchors the integrity chain. The `PayloadVersion` allows future evolution of a payload layout without allocating a new block type. The payload-specific fields described in Section 5 follow immediately at file offset `0x50`.

4.2 Block Flags

The 16-bit `Flags` field at block offset `0x06` carries per-block metadata. The defined flags are listed in Table 4.

Table 4: Block flags (16-bit at `0x06`).

Bit(s)	Name	Description
0	`Compressed`	Algorithm in bits 1–2.
1–2	`CompAlgo`	00=none, 01=zstd, 10=lz4, 11=reserved.
3	`HasKeyHints`	One or more Key Hint blocks (`0x0020`) reference this block.
4	`HasChildren`	Referenced via `ParentUUID`.
5	`Continuation`	This block is a continuation fragment of a larger logical unit (e.g., a memory region >4 GiB). `ParentUUID` references the first fragment.
6–15	`Reserved`	Zero.

CompAlgo semantics. If the `Compressed` flag (bit 0) is not set, a consumer **MUST** ignore bits 1–2 and treat the payload as uncompressed. A producer **MUST** set bits 1–2 to `00` when the `Compressed` flag is not set. The value `11` is reserved for future "extended" use: the compression algorithm identifier would then appear in the first 2 bytes of the payload.

4.3 Block Type Registry

Block types are partitioned into two namespaces (Table 5), separating capture-time data from structural navigation aids. Acquirers write capture-time blocks; analysis tools that need to persist navigational metadata (e.g., a reconstructed virtual address space map) **MAY** append Structural blocks. Analysis results (structure candidates, annotations, scan signatures) are outside this format's scope and belong in the analysis tool's own output.

Table 5: Block type namespaces.

Range	Namespace	Writer	Examples
`0x0000`–`0x0FFF`	Capture-Time	Acquirer/Importer	Regions, modules, threads, keys, import provenance
`0x1000`–`0x1FFF`	Structural	Analysis tool	VAS map, pointer graph (navigational aids only)
`0x2000`–`0xFFFF`	*Reserved*	—	Reserved for future specification versions.

Table 6: Defined block types.

Type	Name	Description
0x0000	*Invalid*	Reserved. **MUST NOT** appear in a valid file. Enables detection of zero-filled gaps.
0x0001	Memory Region	Per-page memory (Sec. 5.1).
0x0002	Module Entry	Module metadata (Sec. 5.2).
0x0010	Module List Index	Module manifest with pre-assigned UUIDs (Sec. 5.3). **MUST** be Block 1 for live acquisition.
0x0011	Thread Context	Register state.
0x0012	File Descriptor	Open handle.
0x0013	Network Connection	Socket attribution.
0x0014	Environment Block	Env vars.
0x0015	Security Token	Credentials.
0x0020	Key Hint	Cryptographic key location annotation (Sec. 5.6).
0x0030	Import Provenance	Raw dump metadata (Sec. 9).
0x0040	Process Identity	Executable path, command line, PPID (Sec. 5.4).
0x0041	Related Dump	Cross-file reference to another MSL dump (Sec. 5.5).
0x0FFF	End-of-Capture	Completeness marker and acquisition end time (Sec. 4.5).
0x1001	VAS Map	Reconstructed virtual address space.
0x1003	Pointer Graph	Pointer relationships between regions.

4.4 Integrity Chain (PrevHash)

The chain uses BLAKE3 [3] with 32-byte (256-bit) output. BLAKE3 was selected for its throughput—typically 3–4× faster than SHA-256 on modern CPUs with SIMD support—which minimizes acquisition overhead. Its collision resistance (128-bit birthday bound) is equivalent to SHA-256; the choice is motivated by performance, not security margin. On platforms without SIMD (e.g., embedded ARM), producers **MAY** observe reduced throughput; this does not affect the security properties of the chain.

- **Block 0:** `PrevHash` $= BLAKE3$(File Header, all `HeaderSize` bytes).
- **Block** i **(**$i \geq 1$**):** `PrevHash` $= BLAKE3$(Block$_{i-1}$, all `BlockLength` bytes).

Wire-format hashing. The `PrevHash` is always computed over the *raw on-disk bytes* of the preceding element—the exact byte sequence as stored in the file. If a block's payload is compressed (per the `Compressed` flag), the hash covers the compressed wire format, not the decompressed content. This ensures that integrity verification requires only sequential byte reads without decompression.

Why hash the previous element?

A block cannot contain its own hash (circular dependency). Each element commits to its predecessor. The chain is transitively anchored: modifying any block in the middle invalidates all subsequent hashes.

4.5 End-of-Capture Block (0x0FFF)

The forward-linked hash chain has an inherent limitation: the last block in the file is not hashed by any successor. More importantly, if the acquirer crashes during capture, there is

no in-file indicator that the dump is incomplete. The End-of-Capture (EoC) block serves as a **completeness marker**: its presence confirms that the acquirer finished normally, and its payload (Table 7) records the acquisition end time. An acquirer **SHOULD** emit the EoC as the final block after all other capture-time blocks have been written.

Table 7: End-of-Capture payload.

Off	Size	Field	Description
+0x00	32	FileHash	BLAKE3 digest of all bytes from file offset 0 through the last byte before this EoC block.
+0x20	8	AcqEnd	Acquisition end time (UTC, nanoseconds since Unix epoch).
+0x28	8	Reserved	**MUST** be zero.

Completeness and corruption detection. If the EoC block is present, a consumer knows the acquirer finished normally. The `FileHash` enables detection of accidental corruption in the capture data (step 1: verify `PrevHash` chain; step 2: verify `FileHash` against computed digest). If the EoC is absent, the dump may be incomplete (acquirer crash) and should be treated accordingly. Note that the `FileHash`, like the `PrevHash` chain, uses an unkeyed hash and does not provide tamper resistance against an adversary who can rewrite the file (see Section 2.2).

If the acquirer crashes during capture, the EoC block is simply absent—alerting the analyst that the dump is incomplete. Analysis tools that append enrichment blocks after the EoC do not invalidate `FileHash`: it covers only the original capture data, and the `PrevHash` chain continues to protect the appended blocks individually.

Implementation guidance: streaming `FileHash` computation

Computing BLAKE3 over the entire file up to the EoC offset can require re-reading multi-GiB dumps if not planned ahead. Producers **SHOULD** maintain a running BLAKE3 hasher state during acquisition, feeding each written byte (file header and all blocks) into the incremental hasher. When writing the EoC block, the producer finalizes the hasher to obtain `FileHash` without a second pass over the file.

With the block architecture, type registry, and integrity chain defined, the next two sections specify the actual payload content: how process memory and module metadata are encoded within their respective block types.

5 Capture-Time Payloads

Payload data begins at byte `0x50` within each block (immediately after the 80-byte Common Block Header). All field offsets in this section are **relative to the payload start** unless otherwise noted.

Each block has exactly **one type**. The `BlockType` field in the Common Block Header determines which payload layout follows at offset `0x50`. For example, a block with `BlockType` = `0x0001` always contains a Memory Region payload (Section 5.1); a block with `BlockType` = `0x0002` always contains a Module Entry payload (Section 5.2). If the same virtual address range is both captured as raw memory and described with module metadata, these are two separate blocks—linked by cross-referencing (Section 7), not merged into one.

5.1 Memory Region (`0x0001`)

A Memory Region block stores the raw byte content of a contiguous range of virtual address space—the actual memory pages that an analysis tool will examine for structures, strings, and key material. Each captured region is stored as a separate block (Table 8, Figure 4). An acquirer **SHOULD** emit one block per contiguous virtual address range as enumerated by the operating system (e.g., via `VirtualQuery` on Windows or `/proc/pid/maps` on Linux). Because acquisition is not atomic, individual pages within a region may become unreadable or unmapped between enumeration and read time; the `PageStateMap` records this per-page outcome (see Section 6).

Table 8: Memory Region payload.

Off	Size	Field	Description
+0x00	8	BaseAddr	Virtual address of the region's first byte.
+0x08	8	RegionSize	Region size in bytes. **MUST** be a multiple of PageSize.
+0x10	1	Protection	Permission and state bits. Bit 0=R, 1=W, 2=X, 3=Guard, 4=CopyOnWrite. Bits 5–7 reserved, zero.
+0x11	1	RegionType	Region origin: 0x00=Unknown, 01=Heap, 02=Stack, 03=Image, 04=MappedFile, 05=Anonymous, 06=SharedMem, FF=Other.
+0x12	1	PageSizeLog2	Page granularity as a power-of-two exponent: actual page size is $2^{\texttt{PageSizeLog2}}$ bytes. Value 12 = 4 KiB; 21 = 2 MiB; 30 = 1 GiB. **MUST** be in the range $[10, 40]$; values outside this range are reserved and **MUST** cause rejection. Per-region to support mixed page sizes.
+0x13	5	Reserved	**MUST** be zero.
+0x18	8	Timestamp	Time this region was acquired (UTC, nanoseconds since Unix epoch). Regions are read sequentially, so timestamps may differ by seconds.
+0x20	var	PageStateMap	2 bits per page: 00=Captured, 01=Failed, 10=Unmapped (Section 6). Padded to 8-byte boundary. Length is derived from RegionSize and PageSize; see below.
$+m$	var	PageData	Concatenated bytes of Captured pages only. Failed and Unmapped pages contribute 0 bytes.

PageStateMap size. The map length is not stored explicitly—it is computed from RegionSize and PageSizeLog2. Let PageSize $= 2^{\texttt{PageSizeLog2}}$ and $P = \texttt{RegionSize} \gg \texttt{PageSizeLog2}$ (the page count). The map requires $P \times 2$ bits. In integer arithmetic (C/Rust):

$$\text{map_bytes} = \text{pad8}\left(\frac{P+3}{4}\right) \quad \text{where} \quad \text{pad8}(n) = (n+7)\ \&\ {\sim}7$$

The PageData immediately follows at offset 0x20 + map_bytes.

Regions exceeding BlockLength capacity

BlockLength is a uint32, limiting any single block to $2^{32} - 1$ bytes (≈4.29 GiB) including the 80-byte header. Contiguous virtual address ranges exceeding this limit (e.g., large JVM heaps, database shared buffers, huge mmap() allocations) **MUST** be split into multiple Memory Region blocks. Each **continuation block** covers a sub-range of the original region at the appropriate BaseAddr and **MUST** set the Continuation flag (bit 6) and ParentUUID to the BlockUUID of the first fragment. The first fragment sets the HasChildren flag (bit 5). Consumers reassemble the region by collecting all blocks with the same ParentUUID (or the first fragment itself), sorting on BaseAddr, and concatenating.

Overhead. The PageStateMap adds minimal size: at 2 bits per page, a 1 GiB region (262,144 pages of 4 KiB) requires only a 64 KiB map—0.006% overhead relative to the page data.

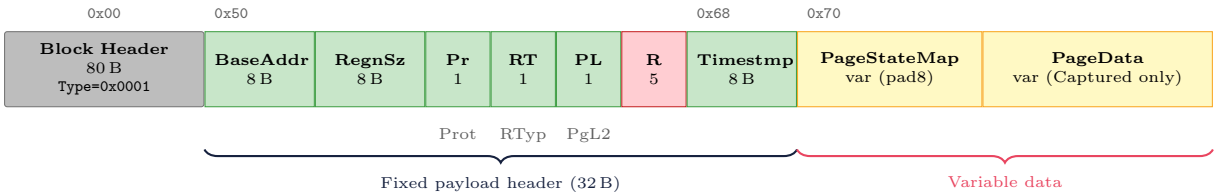

Figure 4: Memory Region (`0x0001`) block layout. Pr=Protection, RT=RegionType, PL=PageSizeLog2, R=Reserved.

5.2 Module Entry (`0x0002`)

While a Memory Region stores raw bytes, a Module Entry stores the *identity and metadata* of a loaded library (DLL, shared object, or dynamic library): its file path, version, on-disk hash, and OS-native loader data. This metadata is essential for interpreting the raw bytes—without knowing which allocator produced a heap region, the byte patterns are opaque. Each loaded module is stored as a separate block whose `ParentUUID` **MUST** reference the Module List Index (type `0x0010`). The `BlockUUID` **MUST** match the pre-assigned `ModuleUUID` from the manifest (Section 5.3). The payload (Table 9) captures both platform-independent fields (path, version, on-disk hash) and an OS-native opaque blob for platform-specific analysis.

Table 9: Module Entry payload.

Off	Size	Field	Description
+0x00	8	BaseAddr	Load address.
+0x08	8	ModuleSize	Mapping size.
+0x10	2	PathLen	Exact byte length of `Path` including the null terminator, *before* padding. The stored (padded) size is pad8(`PathLen`).
+0x12	2	VersionLen	Exact byte length of `Version` including the null terminator, *before* padding. 0 if version is unavailable (no `Version` field emitted). The stored (padded) size is pad8(`VersionLen`).
+0x14	4	Reserved	Zero.
+0x18	var	Path	UTF-8, pad8.
$+p$	var	Version	$p =$ `0x18` + pad8(`PathLen`).
$+v$	32	DiskHash	BLAKE3 of on-disk binary. $v = p +$ pad8(`VersionLen`).
$+v$+32	4	BlobLen	NativeBlob size.
$+v$+36	4	Reserved2	Zero.
$+v$+40	var	NativeBlob	OS-native opaque data.

DiskHash unavailability. If the on-disk binary is inaccessible at acquisition time (e.g., deleted file, `memfd` mapping, insufficient permissions, or fileless malware), the acquirer **MUST** write 32 zero bytes into `DiskHash`. A consumer **MUST** treat an all-zero `DiskHash` as "hash not available" rather than as a valid digest.

Why module metadata matters. The exact version of `ntdll.dll` determines NT Heap chunk layout and XOR key; the exact `libc` determines glibc `malloc` chunk format. Without this information, a heap dump is opaque. The `NativeBlob` provides additional OS-specific context. Producers **SHOULD** populate it as follows: on Windows (`0x0000`), a serialized `LDR_DATA_TABLE_ENTRY`; on Linux (`0x0001`), the corresponding line from `/proc/pid/maps` (or

`smaps`) as a UTF-8 string, null-terminated; on macOS (`0x0002`), a serialized `dyld_image_info` structure; on Android (`0x0003`), the same as Linux plus ART-specific metadata if available. Future versions of this specification may formalize per-OS blob layouts.

5.3 Module List Index (`0x0010`)

An acquirer **MUST** emit the Module List Index as Block 1 (immediately after the Process Identity block; see Figure 5 for the payload layout). The acquirer first enumerates all loaded modules, pre-generates a UUIDv4 for each future Module Entry block, and writes a lightweight **manifest** containing each module's name, load address, and pre-assigned UUID. This allows a consumer to build a complete module map after reading only two blocks—before any Memory Region or Module Entry is encountered.

The full Module Entry blocks (Section 5.2) are emitted later and **MUST** use the pre-assigned UUID from the manifest as their `BlockUUID`. Their `ParentUUID` **MUST** reference this Module List Index block.

Table 10: Module List Index payload.

Off	Size	Field	Description
+0x00	4	EntryCount	Number of module manifest entries.
+0x04	4	Reserved	Zero.
+0x08	var	Entries	EntryCount variable-size entries (Table 11).

Table 11: Module manifest entry (variable-size).

Off	Size	Field	Description
+0x00	16	ModuleUUID	Pre-assigned BlockUUID for the corresponding Module Entry block.
+0x10	8	BaseAddr	Module load address.
+0x18	8	ModuleSize	Module mapping size.
+0x20	2	PathLen	Path length (incl. null).
+0x22	2	Reserved	Zero.
+0x24	4	Reserved2	Zero.
+0x28	var	Path	UTF-8, pad8. Module path/name.

After reading Block 1, a consumer can build: (1) a UUID→module-name map ("UUID `abc...` = `/usr/lib/libssl.so.3`"), (2) an address-range→module map for correlating Memory Regions to their owning library, and (3) a forward-reference table that tells the parser exactly which future blocks correspond to which modules.

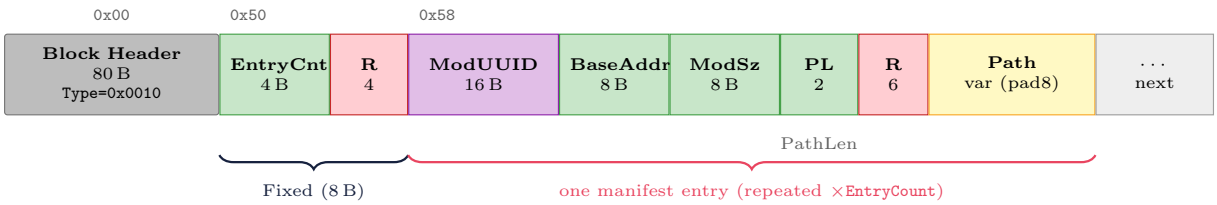

Figure 5: Module List Index (`0x0010`) block layout. PL=PathLen, R=Reserved.

5.4 Process Identity (0x0040)

An acquirer **MUST** emit a Process Identity block as Block 0 (Figure 6), the first capture-time block (or immediately after Import Provenance for imported files). The file header's `PID` identifies the process numerically, but PIDs are ephemeral and recycled; this block provides the context needed to answer "what process is this?" without scanning the entire file. Importers **MAY** omit this block if the source format does not provide the information.

Table 12: Process Identity payload.

Off	Size	Field	Description
+0x00	4	PPID	Parent process ID. 0 if unknown.
+0x04	4	SessionID	Session/login session ID. 0 if unknown or not applicable.
+0x08	8	StartTime	Process start time (UTC, nanoseconds since Unix epoch). 0 if unknown.
+0x10	2	ExePathLen	Length of ExePath (incl. null).
+0x12	2	CmdLineLen	Length of CmdLine (incl. null). 0 if unavailable.
+0x14	4	Reserved	Zero.
+0x18	var	ExePath	UTF-8, pad8. Full executable path.
$+e$	var	CmdLine	UTF-8, pad8. Full command line. $e = \texttt{0x18} + \text{pad8}(\texttt{ExePathLen})$.

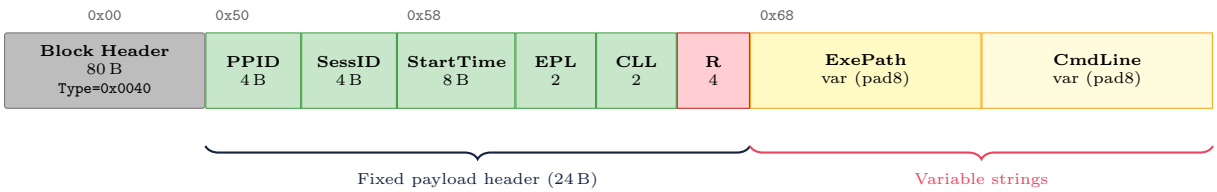

Figure 6: Process Identity (0x0040) block layout, always Block 0 for live acquisitions. EPL=ExePathLen, CLL=CmdLineLen.

5.5 Related Dump (0x0041)

An acquirer **MAY** emit one or more Related Dump blocks to record relationships between concurrently captured processes (e.g., parent/child, shared-memory peers, IPC partners). This enables multi-process forensic workflows where evidence from one dump informs analysis of another.

Table 13: Related Dump payload.

Off	Size	Field	Description
+0x00	16	RelatedDumpUUID	DumpUUID of the related MSL file.
+0x10	4	RelatedPID	PID of the related process. 0 if unknown.
+0x14	2	Relationship	0x0000=Unknown, 0x0001=Parent, 0x0002=Child, 0x0003=SharedMemory, 0x0004=IPC peer, 0x0005=Thread group. 0x0006–0xFFFE: reserved. 0xFFFF=Other.
+0x16	2	Reserved	Zero.

5.6 Key Hint (0x0020)

A Key Hint block (Figure 7) annotates the location and type of cryptographic key material within a captured memory region. This block bridges live and dead forensics: an acquirer that instruments key derivation functions at capture time (e.g., via Frida/friTap) can record where key material resides, so downstream analysis tools need not rediscover it. Key Hint blocks are **OPTIONAL**; their presence is indicated by the CryptoHints bit (bit 10) in CapBitmap and the HasKeyHints flag (bit 3) on the referenced Memory Region block.

Table 14: Key Hint payload.

Off	Size	Field	Description
+0x00	16	RegionUUID	BlockUUID of the Memory Region containing the key material.
+0x10	8	RegionOffset	Byte offset within the region's PageData where the key starts.
+0x18	4	KeyLen	Length of key material in bytes. 0 if unknown.
+0x1C	2	KeyType	Key type code (Table 15).
+0x1E	2	Protocol	Protocol code (Table 15).
+0x20	1	Confidence	0x00=Speculative (region likely contains keys), 0x01=Heuristic (pattern match, e.g., AES key schedule structure), 0x02=Confirmed (captured via KDF instrumentation).
+0x21	1	KeyState	0x00=Unknown, 0x01=Active at capture, 0x02=Expired or zeroed.
+0x22	2	Reserved	Zero.
+0x24	4	NoteLen	Note length (incl. null). 0 if none.
+0x28	4	Reserved2	Zero.
+0x2C	var	Note	UTF-8, pad8. Free-form provenance (e.g., "friTap hook on SSL_do_handshake").

Table 15: Key type and protocol codes for Key Hint blocks.

Key type codes (`KeyType`)

Code	Type
0x0000	Unknown
0x0001	Pre-Master Secret
0x0002	Master Secret
0x0003	Session Key (symmetric)
0x0004	Handshake Secret
0x0005	Application Traffic Secret
0x0006	RSA Private Key
0x0007	ECDH Private Key
0x0008	IKE SA Key
0x0009	ESP / AH Key
0x000A	SSH Session Key
0x000B	WireGuard Key
0x000C--0xFFFE	*Reserved*
0xFFFF	Other

Protocol codes (`Protocol`)

Code	Protocol
0x0000	Unknown
0x0001	TLS 1.2
0x0002	TLS 1.3
0x0003	DTLS 1.2
0x0004	DTLS 1.3
0x0005	QUIC
0x0006	IKEv2 / IPsec
0x0007	SSH
0x0008	WireGuard
0x0009	PQ-TLS (hybrid)
0x000A--0xFFFE	*Reserved*
0xFFFF	Other

Confidence levels. The `Confidence` field (Figure 7) distinguishes three provenance qualities. **Speculative (`0x00`):** the acquirer knows the region belongs to a cryptographic library (e.g., OpenSSL's `.bss`) but has not located specific keys—useful as a search hint for downstream tools. **Heuristic (`0x01`):** a pattern-matching scan (e.g., AES key schedule detection, entropy analysis) identified candidate key material—may produce false positives. **Confirmed (`0x02`):** the acquirer instrumented the application's KDF (e.g., via Frida) and captured the exact key derivation event—this is ground truth with precise offset and length. A dump acquired with such a tool can carry forward its instrumentation knowledge into the MSL file, so a dead-forensics tool can skip discovery and directly extract confirmed keys.

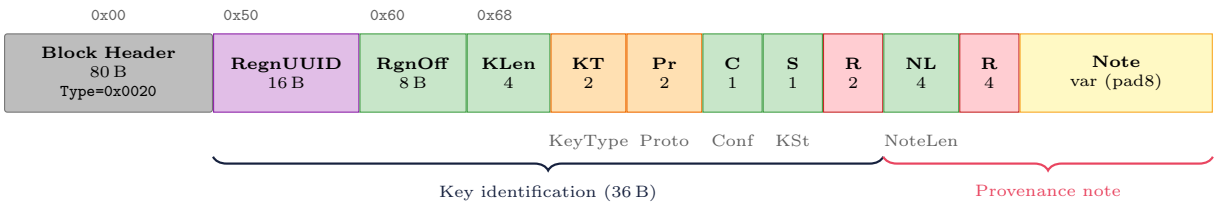

Figure 7: Key Hint (`0x0020`) block layout. KT=KeyType, Pr=Protocol, C=Confidence, S=KeyState, NL=NoteLen, R=Reserved.

6 Three-State Virtual Address Map

The `PageStateMap` in each Memory Region block encodes one of three states for every page using 2 bits per page. Table 16 defines the state codes and Figure 8 shows an example 12-page map.

Table 16: Page states.

Bits	State	Meaning	PageData
00	Captured	Read OK; stored.	PageSize bytes
01	Failed	Mapped but unreadable.	0
10	Unmapped	Not present at read time (TOCTOU; see below).	0
11	*Reserved*	Treat as Failed.	0

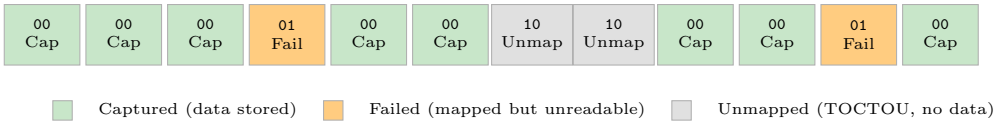

Figure 8: Example PageStateMap for a 12-page region. Only `00` (Captured) pages contribute bytes to PageData.

Why Unmapped exists within a region

Acquisition is not atomic: regions are enumerated first, then pages are read. Between enumeration and read, the process may unmap pages—a Time-of-Check to Time-of-Use (TOCTOU) race condition, where the state observed during enumeration differs from the state encountered during the subsequent read. `Unmapped` records this: the page was in the region at enumeration but absent at read time—distinct from `Failed` (mapped but unreadable) and from never appearing at all.

Reconstruction algorithm. To rebuild the virtual address space from a Memory Region block, a consumer first computes PageSize $= 2^{\texttt{PageSizeLog2}}$, then iterates the `PageStateMap` 2 bits at a time, maintaining a cursor into `PageData`:

1. For page index $i = 0, 1, \ldots, (\texttt{RegionSize}/\text{PageSize}) - 1$:
   a. Read 2 bits from `PageStateMap` at bit position $2i$.
   b. If `00` (Captured): copy PageSize bytes from the cursor into virtual address `BaseAddr` $+ i \times$ PageSize; advance cursor by PageSize.
   c. If `01` (Failed) or `10` (Unmapped): record the state at this address; do *not* advance the `PageData` cursor.

7 Block Cross-Referencing

Blocks in an MSL file do not exist in isolation—they form a graph of relationships that enables consumers to navigate from a discovered structure candidate back to its source memory region and the module whose allocator produced it. The cross-referencing model provides two complementary layers:

Layer 1: `ParentUUID` (Common Block Header). Every block's header includes a `ParentUUID` field that establishes a "belongs-to" hierarchy. Because this field is part of the Common Block Header, any consumer can traverse it without understanding payload layouts. For example, a Module Entry block's `ParentUUID` references the Module List Index block that groups all modules, and a Memory Region block's `ParentUUID` may reference the Module Entry for the library whose image occupies that region.

Layer 2: Payload-embedded UUIDs. For relationships that do not fit a simple parent–child model, individual payload layouts include dedicated UUID fields. A Key Hint block (type `0x0020`), for instance, contains a `RegionUUID` identifying the Memory Region where the cryptographic key material was found. These richer relationships require type-aware parsing.

Figure 9 illustrates both layers for a concrete scenario: two Module Entry blocks (`ntdll.dll`, `kernel32.dll`) reference their Module List Index via `ParentUUID` (solid arrows). A Heap Region references its owning module. A Key Hint block references its source region via a payload-embedded `RegionUUID` (dashed arrow), enabling an analyst to trace cryptographic key material back to the heap that contains it.

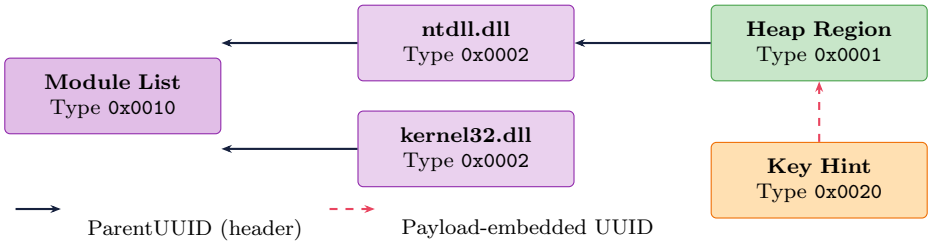

Figure 9: Block cross-referencing: solid = `ParentUUID`, dashed = payload-embedded UUID.

8 Capability Bitmap

The file header's `CapBitmap` (8 bytes, 64 bits) documents which metadata categories the acquirer was able to capture. A consumer **SHOULD** check this bitmap before searching for specific block types. Table 17 lists the defined bits.

Table 17: Capability bits (64-bit `CapBitmap`).

Bit	Name	Description
0	MemoryRegions	Type `0x0001` present.
1	ModuleList	Types `0x0010`/`0x0002` present.
2	ThreadContexts	Type `0x0011`.
3	FileDescriptors	Type `0x0012`.
4	NetworkState	Type `0x0013`.
5	EnvironmentVars	Type `0x0014`.
6	SharedMemory	IPC identifiers.
7	SecurityContext	Type `0x0015`.
8	ProcessIdentity	Type `0x0040` present.
9	RelatedDumps	Type `0x0041` present.
10	CryptoHints	Key Hint blocks (`0x0020`) present.
11–63	Reserved	Zero.

An acquirer **MUST** set exactly those bits corresponding to the metadata categories it successfully captured. Setting a bit without emitting the corresponding blocks, or emitting blocks without setting the bit, are both non-conformant. An importer (Section 9) typically sets only bit 0 (`MemoryRegions`) unless the source format provides additional metadata.

The preceding sections defined the format for live process acquisition—the ideal case where an acquirer captures memory alongside rich OS metadata. In practice, analysts frequently encounter raw process dumps from existing tools that lack this metadata. The next section defines how such dumps can be imported into the MSL format, preserving whatever information the source provides and making the gaps explicit.

9 Raw Dump Import

Raw process dumps (`/proc/pid/mem`, `ReadProcessMemory`, `gcore`, Minidumps) lack forensic metadata. MSL supports importing them into valid MSL files, enabling unified downstream tooling.

9.1 Import Provenance Block (`0x0030`)

An importer **MUST** emit exactly one Import Provenance block as the first block (Block 0). The payload (Table 18) records the source format (Table 19), the importing tool's identity, and a free-form note describing any conversion limitations.

Table 18: Import Provenance payload.

Off	Size	Field	Description
+0x00	2	SourceFormat	See Table 19.
+0x02	2	Reserved	Zero.
+0x04	4	ToolNameLen	Tool name length (incl. null).
+0x08	8	ImportTime	Import time (nanoseconds since Unix epoch).
+0x10	8	OrigFileSize	Original file size. 0 if unknown.
+0x18	4	NoteLen	Note length (incl. null). 0 if none.
+0x1C	4	Reserved2	Zero.
+0x20	var	ToolName	UTF-8, pad8.
$+t$	var	Note	UTF-8, pad8.

Table 19: Source format codes.

Code	Format	Notes
0x0000	Unknown	
0x0001	Raw byte stream	`/proc/pid/mem`, `ReadProcessMemory`.
0x0002	ELF core dump	`gcore(1)` or kernel core.
0x0003	Windows Minidump	`.mdmp`.
0x0004	macOS core dump	Mach-O core.
0x0005	ProcDump	Sysinternals.
0x0006--0xFFFE	*Reserved*	
0xFFFF	Other	

9.2 Import Procedure

1. **File header:** Set `Flags` bit 0 (`Imported`). Set `OSType/ArchType/PID` if known, else `0xFFFF/0xFFFF/0`. Set `CapBitmap` to reflect only available metadata.
2. **Block 0:** Import Provenance (`0x0030`).
3. **Memory Regions:** For raw streams without addresses: single region, `BaseAddr`=0, all Captured. For ELF cores: parse program headers. For Minidumps: parse `MINIDUMP_MEMORY_LIST`.
4. **Modules:** Extract from source if available (ELF `NT_FILE`, Minidump module list).
5. **Hash chain and EoC:** Block 0's PrevHash = $BLAKE3$(File Header); subsequent blocks chain normally. Emit an End-of-Capture block (`0x0FFF`) as the final block.

What import preserves and what it loses

Gains: standardized container, three-state page map, UUID cross-referencing, enrichment capability. **Retains:** whatever the source provided (ELF program headers, Minidump module lists). **Cannot recover:** live FD tables, network attribution, precise module versions (unless in source), IPC identifiers, security tokens. The `CapBitmap` and Import Provenance make these gaps explicit.

The specification has now defined all structural elements: file header, block architecture, integrity chain, payload layouts, page state model, cross-referencing, capability bitmap, and raw dump import. The remaining sections provide practical guidance: a step-by-step parsing walkthrough, a fully worked byte-level example, and the formal conformance requirements for producers and consumers.

10 Parsing Walkthrough

Step 1: File header.
Read 9 bytes. Verify magic. `Endianness` at `0x08`. `HeaderSize` at `0x09`. Read remaining header in determined byte order.

Step 2: Blocks.
Start at `HeaderSize`. Read 80B header, verify block `Magic` (`0x4D534C43`), parse or skip per `BlockType`/`BlockLength`. Continue until fewer than 80 bytes remain or the next 4 bytes do not match the block magic.

Step 3: Integrity chain (OPTIONAL but recommended).
Compute $BLAKE3$(header), verify against Block 0's `PrevHash`. For each block $i \geq 1$, compute $BLAKE3(\text{block}_{i-1})$ over its raw on-disk bytes and compare against block i's `PrevHash`.

Step 4: EoC verification.
If an End-of-Capture block (`0x0FFF`) is present, compute $BLAKE3$ over all file bytes preceding it and verify against `FileHash`. If absent, the dump may be incomplete or the acquirer did not emit one; consider external digest verification.

Step 5: Index.
`BlockUUID`→offset, `ParentUUID`→children.

Step 6: Import check.
`Flags` bit 0 set? Read Import Provenance (`0x0030`).

Step 7: Payloads.
Regions via PageStateMap. Modules via pad8 offsets.

11 Worked Example

Table 20 shows a complete Module Entry block for `ntdll.dll` version `10.0.22621.1` at `0x7FFDD4D00000`, in a little-endian file.
Sizes: Path=30B (pad8=32). Version=13B (pad8=16). Payload: 8+8+2+2+4+32+16+32+4+4+384 = 496B. Block: 80+496 = 576 (`0x0240`).

Table 20: ntdll.dll Module Entry.

Off	Field	Hex	Value
— Header (80B) —			
0x00	Magic	4D 53 4C 43	MSLC
0x04	BlockType	02 00	Module Entry
0x08	BlockLength	40 02 00 00	576
0x0C	PayloadVersion	01 00	1
0x0E	Reserved	00 00	
0x10	BlockUUID	(16B)	UUIDv4
0x20	ParentUUID	(16B)	Module List UUID
0x30	PrevHash	(32B)	BLAKE3 of prev
— Payload (496B) —			
0x50	BaseAddr	00 00 D0 D4 FD 7F 00 00	0x7FFDD4D00000
0x58	ModuleSize	00 00 20 00 00 00 00 00	2 MiB
0x60	PathLen	1E 00	30
0x62	VersionLen	0D 00	13
0x68	Path (32B)	43 3A 5C 57...	ntdll.dll
0x88	Version (16B)	31 30 2E 30...	10.0.22621.1
0x98	DiskHash (32B)	...	BLAKE3 of binary
0xB8	BlobLen	80 01 00 00	384
0xC0	NativeBlob	(384B)	LDR_DATA_TABLE_ENTRY

The worked example above demonstrates how the specification's rules compose into a concrete block. The final section formalizes the obligations of producers and consumers, distinguishing mandatory requirements from recommendations.

12 Conformance

12.1 Producer

A conformant producer **MUST**: (1) write a valid header with `Magic`, `Endianness` (default `0x01`), `HeaderSize`, `Version`; (2) use byte order per `Endianness`; (3) generate UUIDv4 in RFC 4122 network byte order; (4) compute BLAKE3 `PrevHash`: Block 0 hashes the header, subsequent blocks hash the preceding block over raw on-disk bytes; (5) set reserved fields to zero; (6) pad8 all variable fields; (7) never modify written blocks or the file header after subsequent data is written; (8) set `CapBitmap` accurately; (9) if importing: set `Flags` bit 0 and emit Import Provenance (`0x0030`) as Block 0; (10) set `PayloadVersion` to 1 for all payload layouts defined in this specification version; (11) set `CompAlgo` bits (1–2) to `00` when the `Compressed` flag is not set; (12) split memory regions exceeding $2^{32} - 81$ bytes into multiple blocks with the `Continuation` flag (Section 5.1); (13) write 32 zero bytes into `DiskHash` when the on-disk binary is unavailable; (14) reject `PageSizeLog2` values outside $[10, 40]$.
A conformant acquirer (live capture) **MUST** emit a Process Identity block (`0x0040`) as Block 0 and a Module List Index (`0x0010`) as Block 1 with pre-assigned UUIDs for all enumerated modules. An importer **MAY** omit these if the source format does not provide the information.
A conformant producer **SHOULD** emit an End-of-Capture block (type `0x0FFF`, Section 4.5) as the final capture-time block to mark the acquisition as complete.

12.2 Consumer

A conformant consumer **MUST**: (1) reject files with invalid file magic or `Endianness`; (2) use `HeaderSize` to locate Block 0; (3) skip unknown block types via `BlockLength`; (4) detect end-of-blocks when fewer than 80 bytes remain or the next 4 bytes are not `0x4D534C43`; (5) ignore reserved fields; (6) reject block type `0x0000` as invalid; (7) skip blocks with an unrecognized `PayloadVersion` for a known `BlockType` via `BlockLength`; (8) treat an all-zero `DiskHash` as "hash not available"; (9) ignore `CompAlgo` bits when the `Compressed` flag is not set; (10) reject `PageSizeLog2` values outside $[10, 40]$.
A conformant consumer **SHOULD**: (1) verify the BLAKE3 chain from the file header through all blocks; (2) if an EoC block is present, verify `FileHash` against the computed digest of the file up to the EoC block; (3) check `CapBitmap` before searching for specific block types. If no EoC block is present, the consumer **SHOULD** verify the file against an externally stored digest when available.

References

[1] S. Bradner, "Key words for use in RFCs to Indicate Requirement Levels," RFC 2119, 1997. https://www.rfc-editor.org/rfc/rfc2119

[2] P. Leach, M. Mealling, R. Salz, "A Universally Unique IDentifier (UUID) URN Namespace," RFC 4122, 2005. https://www.rfc-editor.org/rfc/rfc4122

[3] J. O'Connor et al., "BLAKE3: One function, fast everywhere," 2020. https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf

[4] TIS Committee, "ELF Specification," v1.2, 1995.